C++ Standard Template Library (STL)

IF2210 – Semester II 2025/2026

C++ Standard Template Library (STL)

- STL menyediakan implementasi konkret dari konsep *collection* dalam C++.
- Dalam sesi sebelumnya kita melihat bahwa:
  - Collection adalah objek yang mengelola sekumpulan elemen.
  - Collection menjaga konsistensi internalnya.
  - Operasi terhadap collection diekspresikan sebagai message tingkat tinggi.
- STL mewujudkan ide-ide tersebut dalam bentuk library generik.

Collection Sebagai Objek dalam STL

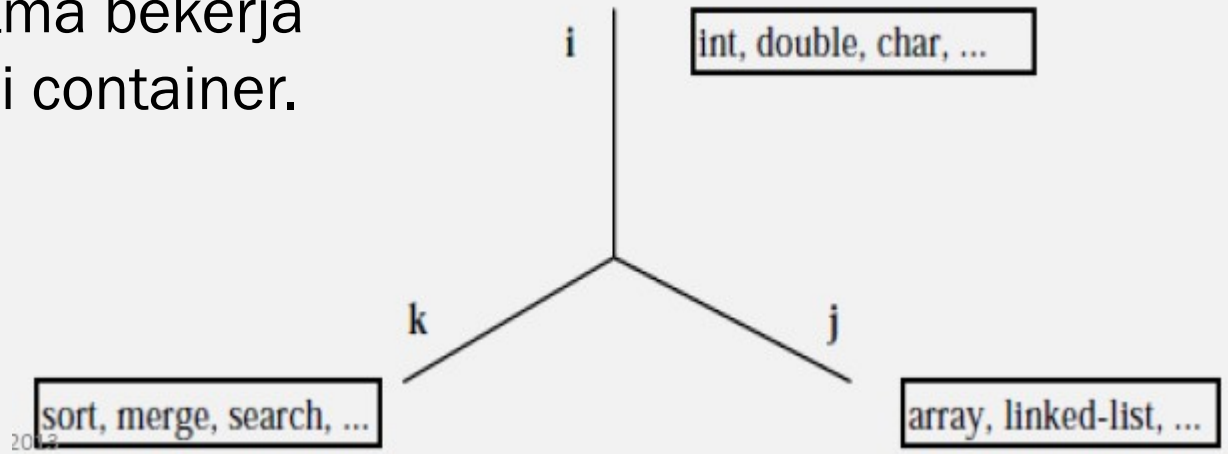
- Dalam STL, collection direpresentasikan sebagai **container**.
- Container:
  - Menyimpan elemen.
  - Mengelola lifecycle elemen.
  - Menjaga invariant internal.
  - Menyediakan operasi melalui method.
- Contoh:

```
std::vector<int> numbers;
numbers.push_back(10);
numbers.push_back(20);
```

- Kita mengirim pesan `push_back` ke objek `vector`.
- Representasi internal dikelola sepenuhnya oleh container.

Tiga Peran dalam STL

- STL memisahkan tanggung jawab menjadi tiga peran ortogonal:
  1. **Container** → menyimpan elemen.
  2. **Iterator** → menyediakan cara mengakses elemen.
  3. **Algorithm** → memproses elemen.
- Pemisahan ini memungkinkan algoritma bekerja tanpa bergantung pada implementasi container.



Container dan Kontrak Perilaku

- Jenis container dibedakan oleh kontrak operasinya.

Sequence container

- Menjaga urutan elemen.
- Mendukung iterasi berurutan.
- Contoh: `vector`, `list`, `deque`.

Associative container

- Menyimpan elemen berdasarkan key.
- Mendukung pencarian berdasarkan key.
- Contoh: `map`, `set`.

Container adapter

- Membatasi operasi agar sesuai model tertentu.
- Contoh: `stack`, `queue`.

Setiap container menawarkan perilaku yang jelas melalui operasinya.

std::vector sebagai Sequence Container

- `std::vector`:
  - Menyimpan elemen dalam urutan.
  - Mendukung akses indeks.
  - Mengelola kapasitas secara otomatis.
- Contoh:

```
std::vector<int> values = {1, 2, 3};
values.push_back(4);
int x = values[0];
```

- Vector menjaga konsistensi strukturnya sendiri.
- Klien bekerja melalui operasi yang tersedia.

Iterator sebagai Abstraksi Akses

- Iterator memungkinkan akses elemen tanpa mengetahui representasi internal container.
- Contoh:

```
for (auto it = values.begin(); it != values.end(); ++it) {
    std::cout << *it << std::endl;
}
```

- `begin()` dan `end()` adalah bagian dari kontrak minimal container.
- Iterator menjadi jembatan antara container dan algoritma.

Algoritma Generik

- Algoritma STL bekerja berdasarkan kontrak iterator.
- Contoh:

```
std::sort(values.begin(), values.end());
```

- Algoritma:
  - Tidak bergantung pada jenis container.
  - Bergantung pada kemampuan iterator.
  - Bekerja untuk berbagai tipe elemen.
- Ini adalah bentuk parameterisasi tipe melalui template.

Generic Programming dalam STL

- Template memungkinkan algoritma bekerja untuk berbagai tipe.
- Selama tipe mendukung operasi yang diperlukan, algoritma dapat digunakan.
- Contoh:

```
std::vector<double> data = {3.1, 2.4, 5.6};
std::sort(data.begin(), data.end());
```

- Template memungkinkan reuse tanpa membentuk hierarki kelas.

Operasi Tingkat Tinggi pada Collection

- STL menyediakan operasi yang mengekspresikan intent.
- Contoh:

```
auto it = std::find(values.begin(), values.end(), 42);
int count = std::count_if(values.begin(), values.end(),
        [](int x){ return x % 2 == 0; });
```

- Kita bekerja pada level:
  - “Temukan elemen”
  - “Hitung elemen yang memenuhi kondisi”
  - Bukan pada level manipulasi indeks.

Container Adapter

- Container adapter membungkus container lain dan membatasi operasinya.
- Contoh:

```
std::stack<int> s;
s.push(10);
s.pop();
```

- `stack` merepresentasikan model LIFO.
- Perilaku didefinisikan oleh kontraknya, bukan oleh struktur internalnya.

Invariant dan Exception Safety

- Container STL:
  - Menjaga konsistensi internal.
  - Melempar exception ketika kontrak dilanggar.
- Contoh:

```
values.at(10); // melempar exception jika indeks tidak valid
```

- RAII memastikan resource tetap aman ketika terjadi exception.

Kolaborasi Container dan Algoritma

- Container menyediakan data.
- Algoritma menyediakan pemrosesan.
- Contoh:

```
std::vector<int> nums = {1,2,3,4,5};
std::reverse(nums.begin(), nums.end());
```

- Kolaborasi ini terjadi melalui iterator sebagai kontrak bersama.

STL sebagai Desain Berbasis Kontrak

- STL menunjukkan bahwa:
  - Struktur data dan algoritma dapat dipisahkan.
  - Operasi bergantung pada kontrak perilaku.
  - Parameterisasi tipe memungkinkan generalisasi.
- Desain ini mendukung keterbacaan dan modularitas.

Contoh-Contoh

vector (1)

- `vector`
  - **Header** `<vector>`
  - Sequence container yang menjaga urutan elemen dan menyediakan akses indeks, sambil mengelola penyimpanan secara otomatis.
    - Akses elemen dengan `v[i]` atau `v.at(i)`
  - Digunakan jika data harus diurutkan dan data harus mudah diakses
- Saat kapasitas internal tidak cukup, vector dapat melakukan **reallocation**: menyiapkan penyimpanan baru dan memindahkan elemen.
  - Konsekuensinya, iterator/reference/pointer ke elemen bisa menjadi tidak valid.
  - gunakan `v.reserve(n)` jika perkiraan ukuran sudah diketahui.

vector (2)

- Deklarasi:
  - `std::vector<type> v;`
    - *type*: `int`, `float`, etc.
  - Iterator:
    - `auto it = v.begin();  // iterator dari depan`
    - `auto it = v.cbegin(); // const. iterator (tidak dapat mengubah`
      `// elemen`
    - `for (auto it = v.rbegin(); it != v.rend(); ++it)  // reverse`
    - `for (auto it = v.crbegin(); it != v.crend(); ++it) // reverse dan`
      `// const.`

Fungsi-fungsi pada vector (1)

- `std::vector<int> v = {1,2,3};`: inisialisasi literal.
- `v.insert(v.end(), other.begin(), other.end());`: copy `other` ke `v`.
- `v.push_back(value)`: menambah elemen di akhir *vector* (dimiliki oleh semua *container* sekuens).
- `v.size()`: ukuran *vector* saat ini.
- `v.capacity()`: jumlah elemen yang dapat ditampung sebelum realokasi. Realokasi meningkatkan kapasitas sesuai strategi implementasi. Standar tidak menentukan faktor pertumbuhannya.

Fungsi-fungsi pada vector (2)

- `v.insert(iterator, value)`: menambahkan elemen `value` di depan lokasi `iterator`
- `v.erase(iterator)`: hapus elemen dari *container*
- `v.erase(iter1, iter2)`: hapus elemen pada `iter1` hingga sebelum `iter2`
- `v.clear()`: kosongkan *container*
- **Catatan: Iterator Invalidation**
  - `push_back` dapat meng-invalidasi semua iterator jika terjadi reallocation.
  - `insert` dan `erase` dapat meng-invalidasi iterator setelah posisi tersebut.
  - `clear` meng-invalidasi semua iterator.

Fungsi-fungsi pada vector (3)

- `v.front()`, `v.back()`: mengembalikan elemen pertama dan terakhir
  - Yang dikembalikan adalah “lvalue reference”, jadi bisa digunakan sebagai left-hand side, e.g., `v.front() = 5;`
    - Tentunya kecuali `v` dideklarasikan sebagai `const`.
- `v[elementNumber] = value;` : meng-*assign* `value` ke sebuah elemen
- `v.at(elementNumber) = value;` : sama dengan sebelumnya, tapi dengan pemeriksaan indeks. Melempar *exception* `out_of_bounds`

Range-based for

- C++ modern mendukung for-loop yang tidak perlu iterator manual:

```
for (const auto& x : v) {
    std::cout << x;
}
```

- Dibaca: “Untuk setiap objek `x` elemen dari `v`, lakukan `std::cout << x`.”

stack

- `stack`
  - Header `<stack>`
  - Tambah & hapus data dari salah satu ujung saja: LIFO
  - Secara internal dapat menggunakan `vector`, `list`, atau `deque` (*default*)
  - Deklarasi:

```
stack<type, vector<type>> myStack;
stack<type, list<type>> myOtherStack;
stack<type> anotherStack; // default deque
```

    - struktur data internal tidak mempengaruhi *behaviour*, hanya kinerja (`deque` dan `vector` lebih cepat daripada `list`)

Contoh Program

```
// #include <> untuk contoh berikut: algorithm, iostream, numeric, string,
// unordered_map, utility, vector

struct Order {                  // Sample domain: orders with amounts (in cents)
    int amountCents;            // and a simple textual tag.
    std::string tag;            // Using struct (public fields) for quick example;
};                              // in real code, state should be private.

int main() {
    std::vector<Order> orders = {
        {1200, "food"},
        {3500, "transport"},
        { 800, "food"},
        {5000, "electronics"},
        {1500, "food"},
    };
```

```
// ---------------------------------------------------------
// forEach: perform an action for each element
// ---------------------------------------------------------
std::cout << "forEach (print):\n";
std::for_each(orders.begin(), orders.end(), [](const Order& o) {
    std::cout << "  " << o.tag << ": " << o.amountCents << " cents\n";
});

// ---------------------------------------------------------
// filter: select elements that satisfy a predicate
// (STL doesn't have a single "filter" function for containers;
//  common approach is copy_if into a new container)
// ---------------------------------------------------------
std::vector<Order> foodOrders;
std::copy_if(orders.begin(), orders.end(), std::back_inserter(foodOrders),
             [](const Order& o) { return o.tag == "food"; });

std::cout << "\nfilter (tag == food): count=" << foodOrders.size() << "\n";
```

```cpp
// ---------------------------------------------------------
// map: transform elements into another representation
// (common approach is transform into a new container)
// ---------------------------------------------------------
std::vector<int> amounts;
amounts.reserve(orders.size());
std::transform(orders.begin(), orders.end(), std::back_inserter(amounts),
               [](const Order& o) { return o.amountCents; });
std::cout << "\nmap (Order -> amountCents): ";
for (int x : amounts) { std::cout << x << " "; }
std::cout << "\n";

// ---------------------------------------------------------
// reduce: aggregate values into a single result
// (accumulate is the classic STL tool)
// ---------------------------------------------------------
int totalCents = std::accumulate(orders.begin(), orders.end(), 0,
                                 [](int acc, const Order& o) {
                                     return acc + o.amountCents;
                                 });
std::cout << "\nreduce (sum amounts): " << totalCents << " cents\n";
```

```
// ---------------------------------------------------------
// any: does any element satisfy predicate?
// ---------------------------------------------------------
bool anyLarge = std::any_of(orders.begin(), orders.end(),
                            [](const Order& o) { return o.amountCents >= 4000; });

std::cout << "\nany (amount >= 4000): " << (anyLarge ? "true" : "false") << "\n";

// ---------------------------------------------------------
// all: do all elements satisfy predicate?
// ---------------------------------------------------------
bool allPositive = std::all_of(orders.begin(), orders.end(),
                               [](const Order& o) { return o.amountCents > 0; });

std::cout << "all (amount > 0): " << (allPositive ? "true" : "false") << "\n";

// (often paired with "none")
bool noneNegative = std::none_of(orders.begin(), orders.end(),
                                 [](const Order& o) { return o.amountCents < 0; });

std::cout << "none (amount < 0): " << (noneNegative ? "true" : "false") << "\n";
```

```
    // ------------------------------------------------------------
    // A more domain-like example: "group by tag" + reduce per group
    // (STL doesn't have group_by built-in; a map works well)
    // ------------------------------------------------------------
    std::unordered_map<std::string, int> totalByTag;
    for (const auto& o : orders) {
        totalByTag[o.tag] += o.amountCents; // reduce into buckets
    }

    std::cout << "\nreduce by group (totalByTag):\n";
    for (const auto& [tag, sum] : totalByTag) {
        std::cout << "  " << tag << " -> " << sum << " cents\n";
    }

    return 0;
}
```

Takeaway

- STL merealisasikan konsep collection dalam C++.
  - Container adalah objek yang mengelola elemen.
  - Iterator menyediakan abstraksi akses.
  - Algoritma bekerja berdasarkan kontrak iterator.
  - Template memungkinkan generalisasi tipe.
  - RAII menjaga keamanan resource.
- STL adalah contoh desain generik berbasis kontrak dalam sistem berorientasi objek.